DSA STUDY BLUEPRINT SERIES

72. Queue Data Structure

First-In-First-Out Data Storage Structure

Section 08 • C++ Core Data Structures & Algorithms

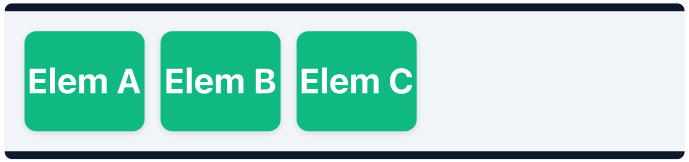
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Queue:** FIFO container structure. Operations: push, pop, front.
- Can be implemented using flat arrays or single linked list structures.

Memory & Execution Blueprint



FIFO Queue conveyor belt (Front = Elem A, Rear = Elem C)

C++ Implementation Reference

Standard code layout and syntax

```
class Queue {
    int arr[1000], fIdx = 0, rIdx = 0;
public:
    void push(int x) { arr[rIdx++] = x; }
    void pop() { fIdx++; }
    int front() { return arr[fIdx]; }
    bool empty() { return fIdx == rIdx; }
};
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
All Operations	O(1) time, O(N) space

Key Practices & Gotchas

- **Important:** Array-based queues suffer from space wastage; use circular pointer wrap-around to recycle memory.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace circular queue pointer wrapping when capacity = 3:

Operation	Queue data layout	Pointers [front, rear]	Size count
enQueue(10)	[10, _, _]	[0, 0]	1
enQueue(20)	[10, 20, _]	[0, 1]	2
deQueue()	[_, 20, _]	[1, 1]	1 (wraps correctly)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Wrap-around indices:** Modulo indexing updates circular buffer limits dynamically.
- **Deque mechanics:** Doubly ended queues allow fast O(1) front and rear pushes and pops.
- **Related LeetCode Problems:** #239 Sliding Window Maximum, #622 Design Circular Queue, #225 Implement Stack using Queues.